



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Swetha Nagasri
Student Name	Myadagoni Ramya Sree
Roll Number	2520030094
Branch	CSE
Course Outcome	CO-1

Case Study Topic: QuizClash Live Tournament Leaderboard using Rank-Augmented AVL Tree.

Work Done Summary:

In this case study, we developed a Rank-Augmented AVL Tree in Java to manage a real-time leaderboard system for an online quiz platform. Player scores were stored in a self-balancing AVL Tree, and each node maintained an additional subtree size field. This size information helps in calculating player ranks efficiently without traversing the entire tree.

Using Java, we implemented score insertion, rank retrieval, and score update operations while maintaining AVL balance through rotations. The subtree size values were updated after every insertion and rotation. The implementation achieved $O(\log n)$ complexity for both score updates and rank queries, making it suitable for large-scale leaderboard applications.

Implementation Details:

1. Created a RankNode class in Java with key, left, right, height, and size fields.
2. Inserted player scores into the AVL Tree using recursive insertion.
3. Updated subtree size and height values after every insertion operation.
4. Performed AVL rotations whenever the balance factor exceeded the allowed limit.
5. Implemented rankOf() method using subtree size augmentation.
6. Verified that score updates and rank queries execute efficiently in $O(\log n)$ time.

Result: Output Screenshots:

```
Inserted Scores:
820 540 910 770 880 460 990 600 730 950 510

Rank of Player Score 770 = 5

Updating Score 540 -> 815
Updating Score 910 -> 685

Updated Leaderboard:
460 510 600 685 730 770 815 820 880 950 990

AVL Operations Completed Successfully
```

The Rank-Augmented AVL Tree was used for efficient leaderboard management and player ranking operations.

Inserted player scores were :

820, 540, 910, 770, 880, 460, 990, 600, 730, 950, 510.

After score update operations :

- Score 540 updated to 815
- Score 910 updated to 685

Updated leaderboard scores :

460, 510, 600, 685, 730, 770, 815, 820, 880, 950, 990.

Rank query result:

Rank of player score **770** = **5**

Time Complexity :

- AVL Tree Insertion: $O(\log n)$
- AVL Tree Deletion: $O(\log n)$
- Score Update (Delete + Insert): $O(\log n)$
- Rank Query: $O(\log n)$
- Top-K Retrieval: $O(\log n + k)$

Performance Analysis :

- Number of active players = 60,000
- AVL Tree Height $\approx \log_2(60000) \approx 16$
- Score Update Cost ≈ 16 operations
- Rank Query Cost ≈ 16 operations
- Supports 10,000 updates/sec efficiently
- Supports 2,000 rank queries/sec efficiently

The Rank-Augmented AVL Tree provides fast insertion, deletion, and ranking operations while maintaining balanced height. Compared to linear searching, AVL Tree significantly improves performance for large-scale real-time leaderboard systems.

GitHub Link:

Student Signature	Faculty Signature